NutriCompass - Architectural Style Suggestions

for the following description,

NutriCompass is a layered supplement evidence analysis system designed to organize and retrieve supplement-related biomedical information under explicitly separated architectural and policy constraints. The system processes supplement inputs obtained through on-device OCR or manual search using a structured pipeline consisting of ingredient extraction, terminology normalization, ontology alignment, adverse-event association retrieval, literature linkage, and evidence aggregation. Architectural responsibilities are separated into distinct processing layers responsible for normalization, ontology mapping, evidence retrieval, aggregation, and policy enforcement in order to reduce cross-layer coupling and preserve controlled responsibility boundaries. NutriCompass integrates multiple large-scale biomedical resources, including the FDA Dietary Supplement Label Database containing 64,444 canonical supplement ingredient entries, PubMed-derived literature indices with 23,562 indexed abstracts, the MeSH 2026 ontology containing 355,016 descriptors and 994,699 entry terms connected to PubMed-indexed literature, and one year of FAERS adverse-event data containing more than 470,000 drug--reaction association records. The datasets are preprocessed into locally indexed SQLite stores supporting ontology-based term expansion, co-occurrence retrieval, semantic lookup, and structured evidence association. Supplement ingredients are matched against indexed relationships linking ingredients, compounds, biomedical terminology, and reported adverse outcomes derived from curated public datasets. AI-related functionality is restricted to OCR processing, entity extraction, semantic matching, terminology alignment, and literature parsing, while policy enforcement is isolated within a dedicated architectural layer that constrains outputs to evidence-linked informational responses without autonomous

medical reasoning or safety judgments. All processing and storage operations are executed locally on-device. By separating semantic processing, evidence aggregation, and policy-controlled output generation into explicitly bounded architectural layers, NutriCompass functions as a structured biomedical evidence-navigation and supplement-information support system.

Suggest suitable architectural styles or patterns for the system. &
Architectural style and pattern selection

Show less

Given the description, **NutriCompass is not well represented by a single architectural style**. The system combines structured biomedical evidence retrieval, ontology-driven knowledge organization, AI-based semantic processing, policy-constrained output generation, and local data management. Consequently, a **combination of architectural styles and patterns** is more appropriate.

Recommended Architectural Styles

1. Layered Architecture (Primary Style)

This should be the dominant architectural style.

Why it fits

The description explicitly separates responsibilities into:

- Input acquisition
- OCR processing
- Ingredient extraction
- Terminology normalization
- Ontology alignment
- Evidence retrieval
- Evidence aggregation
- Policy enforcement
- Persistence

These responsibilities naturally map to architectural layers with controlled dependency relationships.

Possible Layer Structure

Layer	Responsibilities
Presentation	OCR capture, ingredient search, evidence visualization

Layer	Responsibilities
Application	Workflow orchestration and use-case coordination
Domain	Supplement concepts, evidence models, ontology abstractions
AI Processing	OCR, entity extraction, semantic matching, literature parsing
Data	MeSH ontology, FAERS repository, FDA supplement database, PubMed indices
Infrastructure	SQLite stores, indexing services, local persistence
Policy Enforcement	Evidence filtering, response constraints, safety policies

Benefits

- Strong separation of concerns
- Reduced cross-layer coupling
- Explicit responsibility boundaries
- Supports architectural conformance analysis

---

2. Pipe-and-Filter Architecture (Secondary Style)

The evidence-processing workflow closely resembles a pipeline.

Processing Pipeline

```
OCR / Search Input
        ↓
Ingredient Extraction
        ↓
Terminology Normalization
        ↓
Ontology Alignment
        ↓
Evidence Retrieval
        ↓
Evidence Aggregation
        ↓
Policy Enforcement
        ↓
User Output
```

Why it fits

The description explicitly describes sequential transformations:

- ingredient extraction
- terminology normalization
- ontology mapping
- adverse-event retrieval
- literature linkage
- evidence aggregation

Each stage consumes structured output from the previous stage and produces input for the next stage.

Benefits

- High modularity
- Replaceable processing stages
- Easier testing and validation
- Natural fit for biomedical evidence-processing workflows

---

3. Repository Architecture
The system relies heavily on multiple biomedical knowledge sources.

Repositories

- FDA Dietary Supplement Label Database
- PubMed
- MeSH 2026
- FAERS

These datasets collectively function as a shared knowledge repository accessed by multiple processing components.

Benefits

- Centralized knowledge management
- Consistent evidence retrieval
- Simplifies ontology integration
- Supports future dataset expansion

---

4. Policy Enforcement Layer Pattern
A dedicated policy layer is explicitly described.

Why it fits

The system intentionally separates:

- biomedical evidence retrieval

- semantic interpretation
- policy-controlled output generation

The policy layer acts as a gatekeeper between evidence generation and user presentation.

Responsibilities

- Evidence-only response generation
- Output constraint enforcement
- Prevention of autonomous medical reasoning
- Prevention of diagnostic recommendations

Benefits

- Regulatory compliance
- Explainable behavior
- Reduced liability risk
- Clear separation between information retrieval and medical decision making

---

5. Service-Oriented Component Pattern (Within Layers)
Within the layered architecture, several responsibilities are naturally implemented as independent services/components.

Examples include:

Component Service	Responsibility
OCR Service	Text extraction
Ingredient Extraction Service	Ingredient recognition
Normalization Service	Terminology normalization
Ontology Mapping Service	MeSH alignment
Evidence Retrieval Service	Literature and adverse-event lookup
Aggregation Service	Evidence synthesis
Policy Service	Output validation

This improves maintainability without requiring a distributed microservice architecture.

Architectural Styles Less Suitable

Style	Reason
Microservices	System executes locally on-device; distributed deployment is unnecessary
Event-Driven Architecture	Processing is predominantly sequential rather than asynchronous
Client–Server	Core functionality is local and self-contained
Blackboard Architecture	Knowledge integration exists, but processing follows deterministic workflows rather than opportunistic knowledge-source collaboration
Pure MVC	Useful only within the presentation layer and insufficient as the overall system architecture

---

Recommended Combination

The most appropriate architectural solution is:

> **Layered Architecture (primary) + Pipe-and-Filter Processing Pipeline + Repository Architecture + Dedicated Policy Enforcement Layer + Service-Oriented Components within layers.**

This combination directly reflects the system description by supporting explicit responsibility separation, ontology-driven evidence retrieval, controlled dependency relationships, sequential semantic processing, policy-constrained output generation, and local biomedical knowledge management.

For the described NutriCompass system, the architecture can be decomposed into seven layers. The decomposition follows the explicit responsibility boundaries stated in the description and separates user interaction, workflow coordination, biomedical concepts, AI processing, evidence repositories, policy enforcement, and infrastructure concerns.

Layered Architecture Definition

Layer	Composing Components
Presentation Layer	OCR Capture View, Supplement Search View, Ingredient Search Interface, Evidence Browser View, Adverse Event Visualization View, Literature Evidence View, Evidence Summary Dashboard, Search Result Presentation Components, Input Validation Components
Application Layer	Supplement Analysis Controller, OCR Processing Workflow Manager, Ingredient Lookup Coordinator, Evidence Retrieval Coordinator, Ontology Mapping Coordinator, Evidence Aggregation Coordinator, Policy Validation Coordinator, Search Session Manager
Domain Layer	Supplement Entity Model, Ingredient Model, Biomedical Concept Model, Adverse Event Model, Literature Evidence Model, Evidence Association Model, Ontology Relationship Model, Evidence Summary Model, Search Query Model
AI Processing Layer	OCR Engine, Ingredient Extraction Engine, Entity Recognition Component, Terminology Normalization Engine, Semantic Matching Engine, Ontology Alignment Engine, Literature Parsing Component, Evidence Linking Component
Data Layer	FDA Supplement Repository Adapter, MeSH Ontology Repository, PubMed Index Repository, FAERS Repository, Ontology Relationship Index, Ingredient Association Index, Literature Association Index, Adverse Event Association Index
Policy Enforcement Layer	Evidence Validation Engine, Evidence-Only Response Filter, Output Constraint Engine, Citation Verification Component, Safety Constraint Validator, Response Generation Policy Manager
Infrastructure Layer	SQLite Storage Manager, Local Index Manager, Query Engine, Repository Access Layer, Caching Manager, File Storage Manager, Local Persistence Services, Device Resource Manager

Layer Responsibilities

Layer	Primary Responsibility
Presentation	User interaction and evidence visualization
Application	Workflow orchestration and use-case coordination
Domain	Biomedical evidence abstractions and business concepts
AI Processing	OCR, extraction, semantic processing, and ontology alignment

Layer	Primary Responsibility
Data	Management of biomedical datasets and indexed repositories
Policy Enforcement	Constraining outputs to evidence-linked informational responses
Infrastructure	Local storage, indexing, persistence, and runtime support

Architectural Structure

```
Presentation
      ↓
Application
      ↓
Domain
      ↓
AI Processing
      ↓
Data
      ↓
Infrastructure

Policy Enforcement
      ↑
intercepts and validates
outputs produced by
Application and Domain
before presentation
```

This decomposition reflects the system description by isolating biomedical semantic processing within the AI Processing layer, organizing biomedical resources within the Data layer, maintaining explicit workflow coordination in the Application layer, and separating policy-controlled response generation into a dedicated Policy Enforcement layer.

Establish allowed and forbidden dependencies among components across layers. &
Dependency definition and architectural constraint specification

For NutriCompass, dependency constraints should enforce the responsibility boundaries described in the system specification while preventing semantic processing, evidence retrieval, and policy enforcement concerns from becoming intertwined. The resulting dependency model follows a **strict layered architecture with a cross-cutting Policy Enforcement layer**.

Allowed Layer Dependencies

Source Layer	Allowed Dependencies
Presentation	Application, Domain
Application	Domain, AI Processing, Policy Enforcement
Domain	None
AI Processing	Domain, Data
Data	Infrastructure
Policy Enforcement	Domain, Data
Infrastructure	None

Forbidden Layer Dependencies

Source Layer	Forbidden Dependencies
Presentation	AI Processing, Data, Policy Enforcement, Infrastructure
Application	Data, Infrastructure
Domain	Application, AI Processing, Data, Policy Enforcement, Infrastructure
AI Processing	Presentation, Application, Policy Enforcement, Infrastructure
Data	Presentation, Application, Domain, AI Processing, Policy Enforcement
Policy Enforcement	Presentation, Application, AI Processing, Infrastructure
Infrastructure	Presentation, Application, Domain, AI Processing, Data, Policy Enforcement

Dependency Rules by Layer

Presentation Layer

Allowed

- Invoke application workflows.
- Read domain objects for rendering.
- Display evidence summaries and literature associations.

Forbidden

- Direct access to biomedical repositories.
- Direct execution of OCR or semantic matching.
- Direct policy evaluation.
- Direct database access.

Application Layer

Allowed

- Coordinate supplement-analysis workflows.
- Invoke AI-processing services.
- Invoke policy-validation services.
- Consume domain abstractions.

Forbidden

- Direct repository queries.
- Direct SQLite operations.
- Direct access to MeSH, PubMed, FAERS, or FDA datasets.

Domain Layer

Allowed

- Define biomedical concepts and evidence abstractions.

Forbidden

- Dependency on any other layer.
- Database access.
- AI-model invocation.
- UI concerns.
- Policy implementation logic.

The Domain layer should remain the architectural center.

AI Processing Layer

Allowed

- Use domain models.
- Query biomedical repositories through the Data layer.
- Perform OCR, extraction, normalization, ontology alignment, and semantic matching.

Forbidden

- UI interaction.
- Workflow orchestration.
- Policy decisions.
- Direct storage management.

Data Layer
Allowed

- Access local repositories.
- Access indexes and SQLite stores through Infrastructure.

Forbidden

- Workflow logic.
- Semantic reasoning.
- Policy enforcement.
- User-interface concerns.

Policy Enforcement Layer
Allowed

- Inspect domain-level evidence structures.
- Validate evidence provenance.
- Access evidence metadata from repositories.
- Enforce informational-response constraints.

Forbidden

- Execute OCR.
- Perform semantic matching.
- Retrieve evidence independently of the application workflow.
- Render UI.

Infrastructure Layer
Allowed

- Provide persistence, indexing, caching, and storage services.

Forbidden

- Business logic.

- Biomedical reasoning.
- Evidence aggregation.
- Policy decisions.
- User interaction.

Resulting Dependency Structure

Component-Level Constraint Examples

Component	Allowed Dependencies	Forbidden Dependencies
OCR Capture View	Supplement Analysis Controller	OCR Engine, SQLite Storage Manager
Supplement Analysis Controller	Ingredient Model, OCR Engine, Evidence Validation Engine	MeSH Repository, SQLite Storage Manager
OCR Engine	Ingredient Model, FDA Supplement Repository Adapter	Evidence Browser View, Policy Manager
Ontology Alignment Engine	Biomedical Concept Model, MeSH Ontology Repository	Evidence Summary Dashboard
Evidence Aggregation Coordinator	Domain Evidence Models, Evidence Validation Engine	SQLite Storage Manager
Evidence Validation Engine	Evidence Summary Model, Literature Association Index	OCR Engine, OCR Capture View
FDA Supplement Repository Adapter	SQLite Storage Manager	Workflow Managers
SQLite Storage Manager	None	Any higher-layer component

Architectural Constraint Specification

The architecture should satisfy the following rules:

1. **No layer skipping is permitted.**
2. **Presentation components must never access repositories directly.**
3. **Application components must never access persistence services directly.**
4. **Domain components must remain dependency-free.**
5. **AI Processing components may access biomedical repositories only through the Data layer.**
6. **Policy Enforcement components may inspect evidence but must not perform semantic interpretation.**
7. **Infrastructure components must not contain biomedical, workflow, or policy logic.**
8. **All user-visible outputs must pass through Policy Enforcement before presentation.**

These constraints preserve the intended separation between biomedical evidence processing, evidence aggregation, policy-controlled output generation, and local persistence described in the NutriCompass architecture.

Refine the internal structure of each layer using component-level architectural representations.

The layered architecture can be refined further by decomposing each layer into collaborating components and defining their internal relationships. The resulting structure provides a component-level architectural representation that remains consistent with the responsibilities and dependency constraints previously established.

Presentation Layer

Purpose: User interaction, input collection, and evidence visualization.

```
Presentation Layer
├── OCRCaptureView
├── SupplementSearchView
├── IngredientSearchView
├── EvidenceBrowserView
├── AdverseEventVisualizationView
├── LiteratureEvidenceView
├── EvidenceSummaryDashboard
├── SearchResultRenderer
└── InputValidationComponent
```

Internal Relationships

```
OCRCaptureView
        |
SupplementSearchView
        |
IngredientSearchView
        ▼
InputValidationComponent
        ▼
SearchResultRenderer
        ▼
EvidenceBrowserView
├── LiteratureEvidenceView
├── AdverseEventVisualizationView
└── EvidenceSummaryDashboard
```

Application Layer

Purpose: Workflow orchestration and use-case coordination.

```
Application Layer
├── SupplementAnalysisController
├── OCRWorkflowManager
├── IngredientLookupCoordinator
├── OntologyMappingCoordinator
├── EvidenceRetrievalCoordinator
├── EvidenceAggregationCoordinator
├── PolicyValidationCoordinator
└── SearchSessionManager
```

Internal Relationships

```
SupplementAnalysisController
        |
        ▼
OCRWorkflowManager
        |
        ▼
IngredientLookupCoordinator
        |
        ▼
OntologyMappingCoordinator
        |
        ▼
EvidenceRetrievalCoordinator
        |
        ▼
EvidenceAggregationCoordinator
        |
        ▼
```

```
PolicyValidationCoordinator
        │
        ▼
SearchSessionManager
```

Domain Layer

Purpose: Biomedical concepts and evidence abstractions.

```
Domain Layer
├── SupplementEntity
├── IngredientEntity
├── BiomedicalConcept
├── OntologyRelationship
├── LiteratureEvidence
├── AdverseEventEvidence
├── EvidenceAssociation
├── EvidenceSummary
└── SearchQuery
```

Internal Relationships

```
SupplementEntity
        │
        ▼
IngredientEntity
        │
        ▼
BiomedicalConcept
        │
        ▼
OntologyRelationship

BiomedicalConcept
        │
        ├──► LiteratureEvidence
        │
        └──► AdverseEventEvidence

LiteratureEvidence
        │
AdverseEventEvidence
        ▼
EvidenceAssociation
        ▼
EvidenceSummary
```

AI Processing Layer

Purpose: Semantic processing and biomedical information extraction.

```
AI Processing Layer
├── OCREngine
├── IngredientExtractionEngine
├── EntityRecognitionComponent
├── TerminologyNormalizationEngine
├── SemanticMatchingEngine
├── OntologyAlignmentEngine
├── LiteratureParsingComponent
└── EvidenceLinkingComponent
```

Internal Relationships

```
OCREngine
    │
    ▼
IngredientExtractionEngine
    │
    ▼
EntityRecognitionComponent
    │
    ▼
TerminologyNormalizationEngine
    │
    ▼
SemanticMatchingEngine
    │
    ▼
OntologyAlignmentEngine
    │
    ▼
LiteratureParsingComponent
    │
    ▼
EvidenceLinkingComponent
```

This structure mirrors the semantic-processing pipeline described in the system specification.

Data Layer

Purpose: Biomedical repository access and indexed knowledge retrieval.

```
Data Layer
├── FDASupplementRepository
├── MeSHOntologyRepository
├── PubMedIndexRepository
├── FAERSRepository
├── OntologyRelationshipIndex
├── IngredientAssociationIndex
```

```
├── LiteratureAssociationIndex
└── AdverseEventAssociationIndex
```

Internal Relationships

```
FDASupplementRepository
          │
          ▼
IngredientAssociationIndex

MeSHOntologyRepository
          │
          ▼
OntologyRelationshipIndex

PubMedIndexRepository
          │
          ▼
LiteratureAssociationIndex

FAERSRepository
          │
          ▼
AdverseEventAssociationIndex
```

Repositories manage canonical data sources, while indexes support efficient local retrieval.

Policy Enforcement Layer

Purpose: Ensure all outputs remain evidence-linked and policy compliant.

```
Policy Enforcement Layer
├── EvidenceValidationEngine
├── CitationVerificationComponent
├── OutputConstraintEngine
├── SafetyConstraintValidator
└── ResponsePolicyManager
```

Internal Relationships

```
EvidenceValidationEngine
          │
          ▼
CitationVerificationComponent
          │
          ▼
OutputConstraintEngine
          │
          ▼
SafetyConstraintValidator
```

```
          |
          ▼
ResponsePolicyManager
```

The policy-processing sequence ensures that generated outputs remain constrained to evidence-supported informational content.

Infrastructure Layer

Purpose: Local persistence, indexing, and runtime support.

```
Infrastructure Layer
├── SQLiteStorageManager
├── LocalIndexManager
├── QueryEngine
├── RepositoryAccessLayer
├── CacheManager
├── FileStorageManager
├── PersistenceService
└── DeviceResourceManager
```

Internal Relationships

```
RepositoryAccessLayer
          |
          ▼
QueryEngine
          |
          ▼
LocalIndexManager
          |
          ▼
SQLiteStorageManager

CacheManager
      |
      ▼
PersistenceService

FileStorageManager
      |
      ▼
DeviceResourceManager
```

Integrated Component View

```
Presentation
      |
```

```
      ▼
Application
      |
      ▼
Domain
      ▲
      |
AI Processing
      |
      ▼
Data
      |
      ▼
Infrastructure

Application
      |
      ▼
Policy Enforcement
      |
      ▼
Presentation
```

This refinement yields a component-level architecture in which:

- Presentation components manage interaction and visualization.
- Application components orchestrate supplement-analysis workflows.
- Domain components encapsulate biomedical evidence concepts.
- AI Processing components perform OCR, extraction, normalization, alignment, and evidence linking.
- Data components provide access to FDA, MeSH, PubMed, and FAERS repositories.
- Policy Enforcement components constrain outputs to evidence-supported informational responses.
- Infrastructure components provide local storage, indexing, caching, and persistence services.